

Анализ редиректов

Наше облако состоит из множества сайтов, сервисов. Переходы между ними осуществляются через редиректы. Источники и причины редиректов могут быть разные. Анализировать их нужно как на продакшене для решения проблем клиентов, так и на тестовом стенде. В этой статье разбирается методика анализа цепочек перенаправлений через ключевые заголовки ответов, помогающая быстро определить, какая компонента системы инициировала редирект.

Настройка оснастки DevTools браузера

Для анализа редиректов нам нужна оснастка Network из DevTools браузера с выбранным фильтром Doc/Док. Важно также выбрать опцию Preserve log/Сохранять журнал.

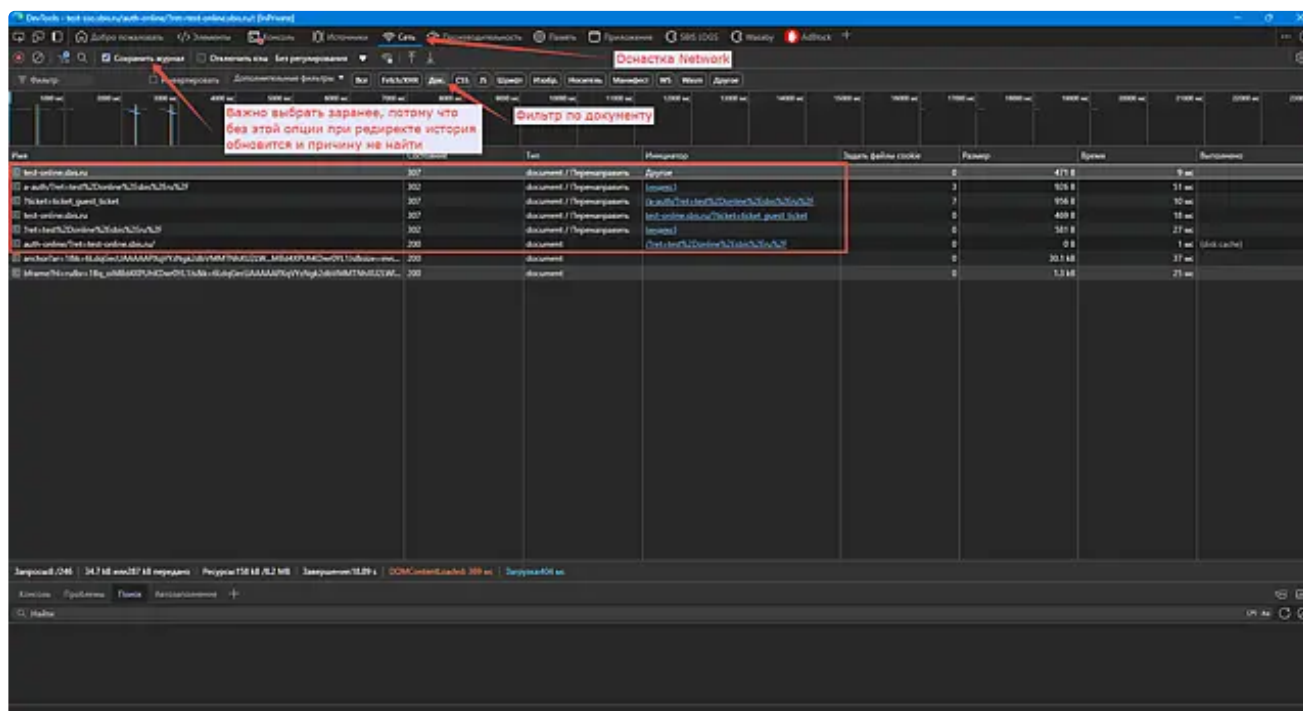


Рис. 1

Инструмент sbislogs для анализа причин редиректов менее удобен, т.к. не содержит полной истории.

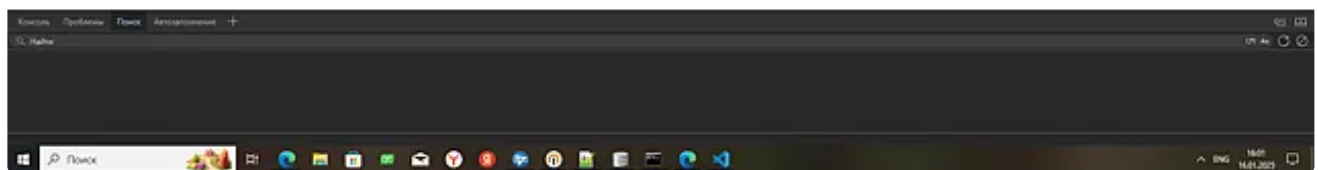


Рис. 2

Разбор источников редиректов

Можно выделить 4 основных источника редиректов:

1. Диспетчер
2. Сервис бизнес-логики
3. Сервис представления
4. JS-код

Рассмотрим все 4 случая на примере входа в систему на стенде test-online.sbis.ru. В Network можно увидеть следующие запросы:

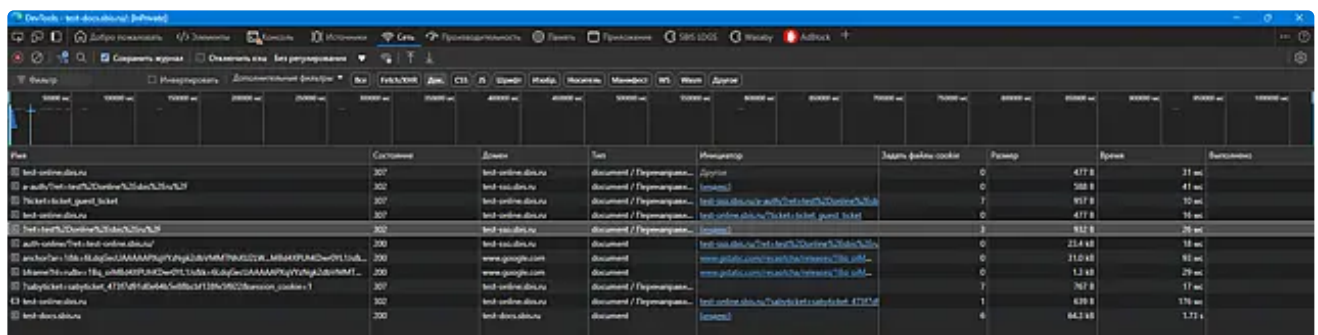


Рис. 3

Опишем последовательно все запросы с редиректами, причины и их источники.

1. Пользователь в браузере вводит <https://test-online.sbis.ru>. Запрос проходит через диспетчер, который проверяет авторизационные данные. В данном случае их нет. Диспетчер отвечает кодом 307 и в заголовке Location передает url, куда браузер должен перенаправить запрос. В примере <https://test-sso.sbis.ru/a-auth/?ret=test%2Donline%2Esbis%2Eru%2F>. Корневой сервис онлайн поддерживает гостевую аутентификацию, поэтому в url появился **a-auth**. Также в заголовках ответа присутствуют важные хедеры:
 - x-dispatch: online-sbis-ru-dispatcher-7579f85b6b-fx2nw (test-online.sbis.ru) - диспетчер, который обработал запрос
 - x-path: ()[notarget-N] - означает, что запрос обработал сам диспетчер
 - x-reason: Set guest [no sid] - причина редиректа (отсутствие сессии)
 - x-reason-rc: 2027 - код причины редиректа

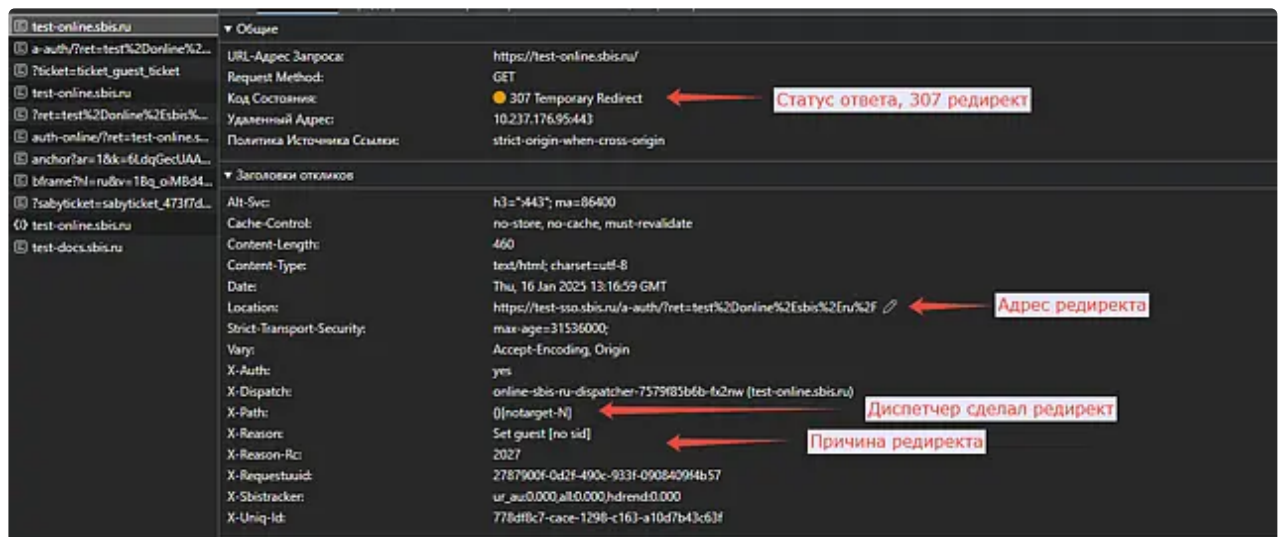


Рис. 4

Данный редирект совершил диспетчер.

2. <https://test-sso.sbis.ru/a-auth/?ret=test%2Donline%2Esbis%2Eru%2F> запрос на sso за гостевой сессией. На sso перенаправил диспетчер. Запрос по этому адрес был обработан сервисом БЛ sso-service. Сервис не смог аутентифицировать пользователя по имеющимся в cookie данным и выдал гостевой тикет на адрес, который пришел в параметре ret. location: https://test-online.sbis.ru/?ticket=ticket_guest_ticket

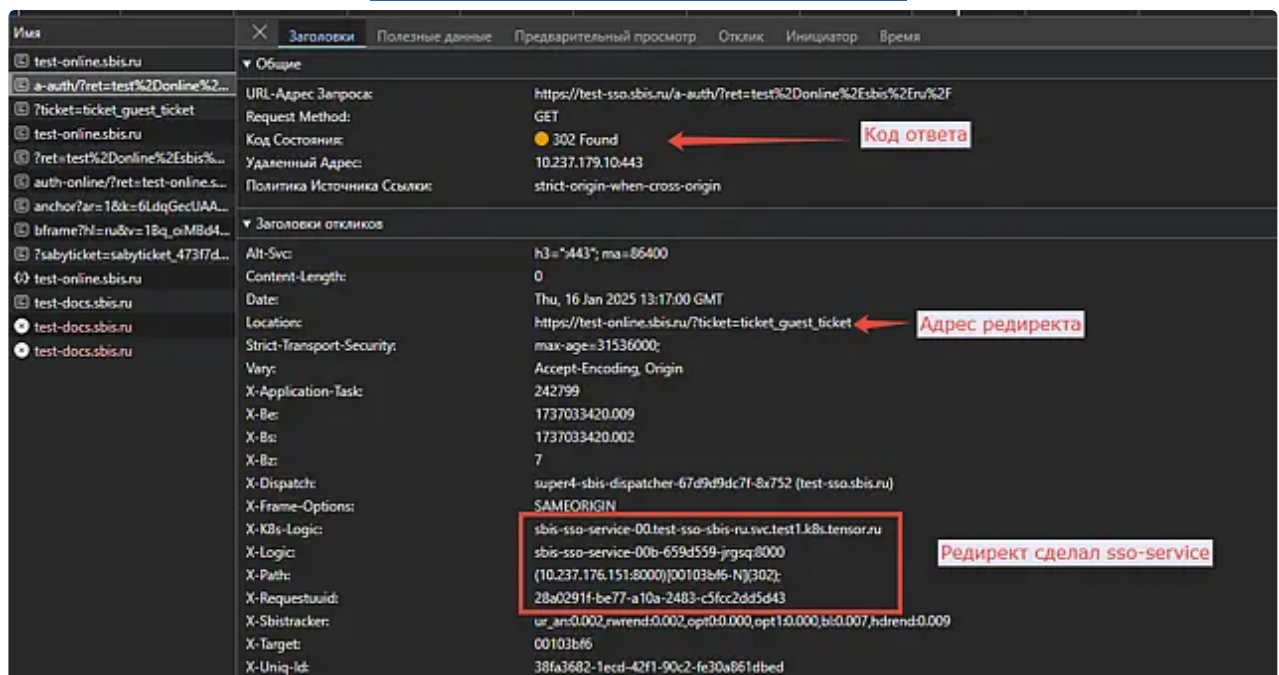


Рис. 5

Данный редирект совершил сервис БЛ sso-service.

3. Следующий запрос с гостевым тикетом https://test-online.sbis.ru/?ticket=ticket_guest_ticket обработал диспетчер. Смысл редиректа на тот же ресурс в простановке cookie аутентификации через механизм тикетов.

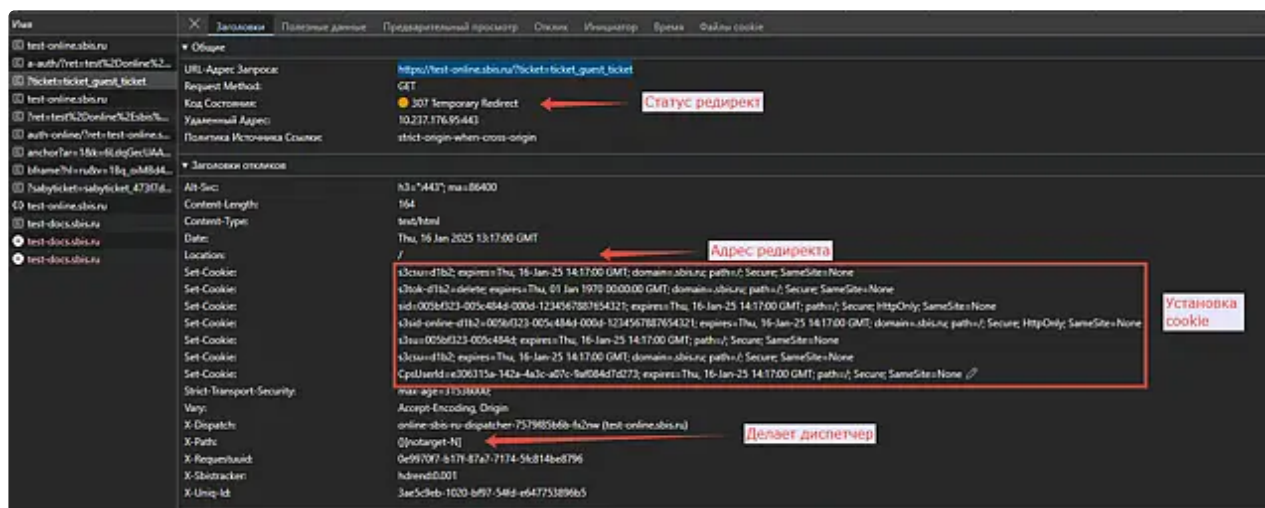


Рис. 6

4. Запрос <https://test-online.sbis.ru/> обрабатывает rs-сервис. Корневой ресурс онлайн недоступен гостю, и rs-сервис при проверке отдает 401-ый код ошибки и заголовок `x-tosso: 1`. Диспетчер модифицирует этот ответ и делает редирект на <https://test-sso.sbis.ru/?ret=test%2Donline%2Esbis%2Eru%2F>. Стоит отметить, что в url отсутствует `a-auth`, потому что гостевая сессия уже выдана, и с ней нельзя получить доступ на текущий ресурс.

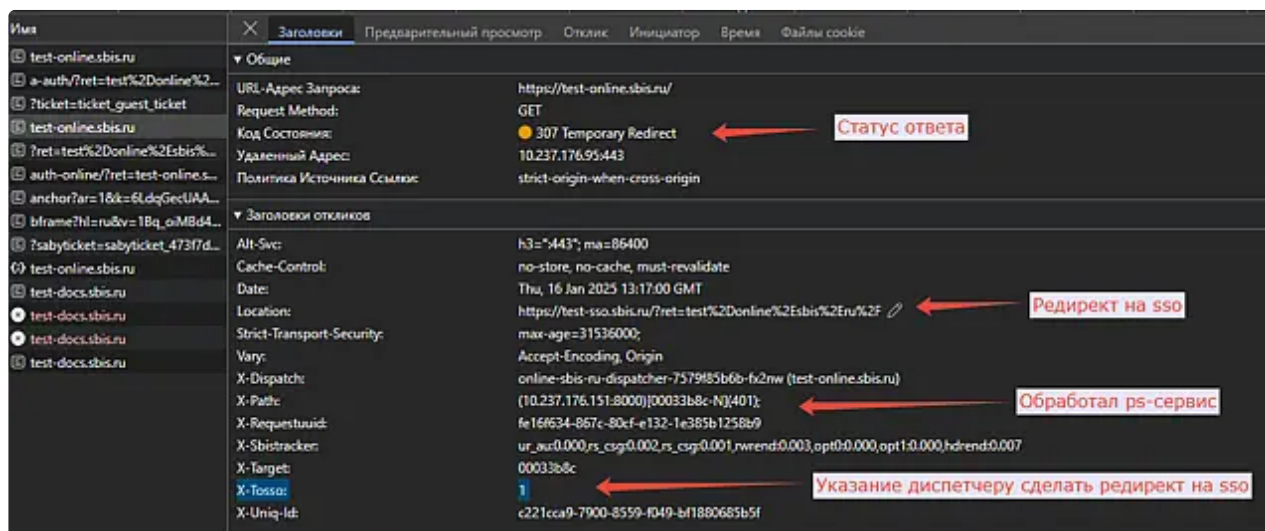


Рис. 7

Запрос обработал rs-сервис, проставил заголовок `x-tosso:1`. Диспетчер обработал этот ответ и сделал редирект на sso.

5. Запрос <https://test-sso.sbis.ru/?ret=test%2Donline%2Esbis%2Eru%2F> обрабатывает сервис БЛ sso-service. Он делает редирект на страницу аутентификации.

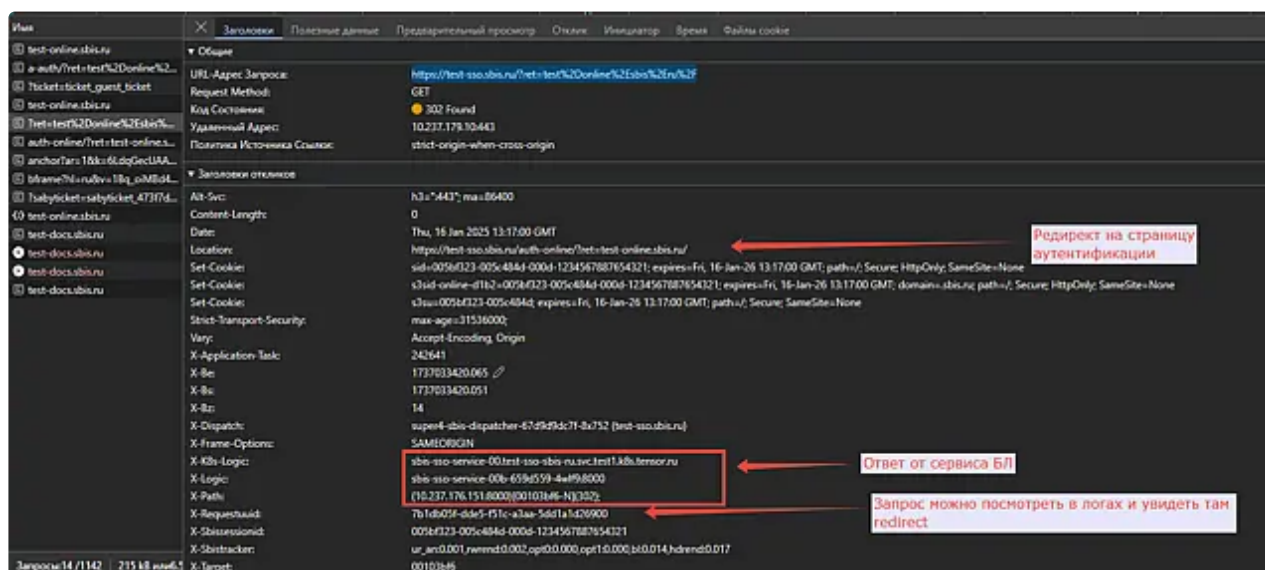


Рис. 8

6. Url <https://test-sso.sbis.ru/auth-online/?ret=test-online.sbis.ru/> это страница аутентификации. Она никуда не редиректит, а предоставляет пользователю форму для ввода идентификационных данных.
7. Пользователь вводит логин пароль, нажимает на кнопку входа. На странице вызывается js-код, который вызывает метод аутентификации и обрабатывает ответ. В качестве ответа js-код получает ссылку с одноразовым тикетом и делает редирект на полученный url. Пример: https://test-online.sbis.ru/?sabyticket=sabyticket_473f7d91d0e64b5e88bcbf138fe5f922&session_cookie=1. Это единственный случай в нашем примере, где переход реализован на клиенте в js-коде.
8. Адрес https://test-online.sbis.ru/?sabyticket=sabyticket_473f7d91d0e64b5e88bcbf138fe5f922&session_cookie=1 содержит тикет, поэтому обрабатывается диспетчером.

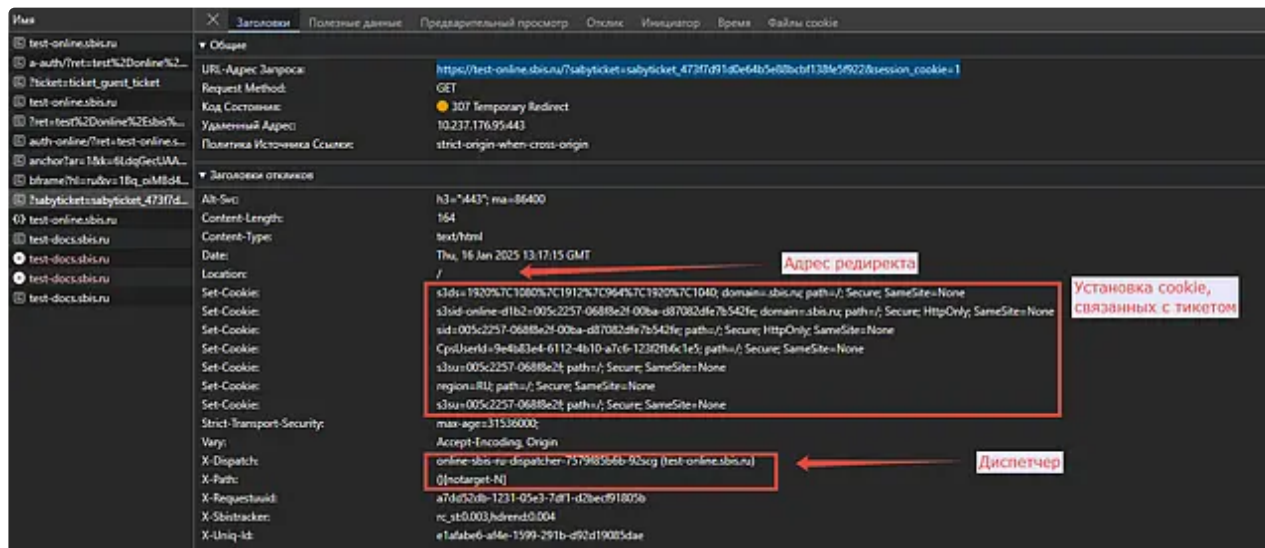


Рис. 9

9. Запрос <https://test-online.sbis.ru/> обрабатывает ps-сервис основного сервиса. Для построения страницы вызывается ряд методов. Один из методов запрашивает конфигурацию приложений для текущего пользователя. Сервис appconfiguration определяет, что текущему пользователю назначена роль, для которой определено основное приложение docs. На основе этой информации ps-service делает редирект на test-docs.sbis.ru.

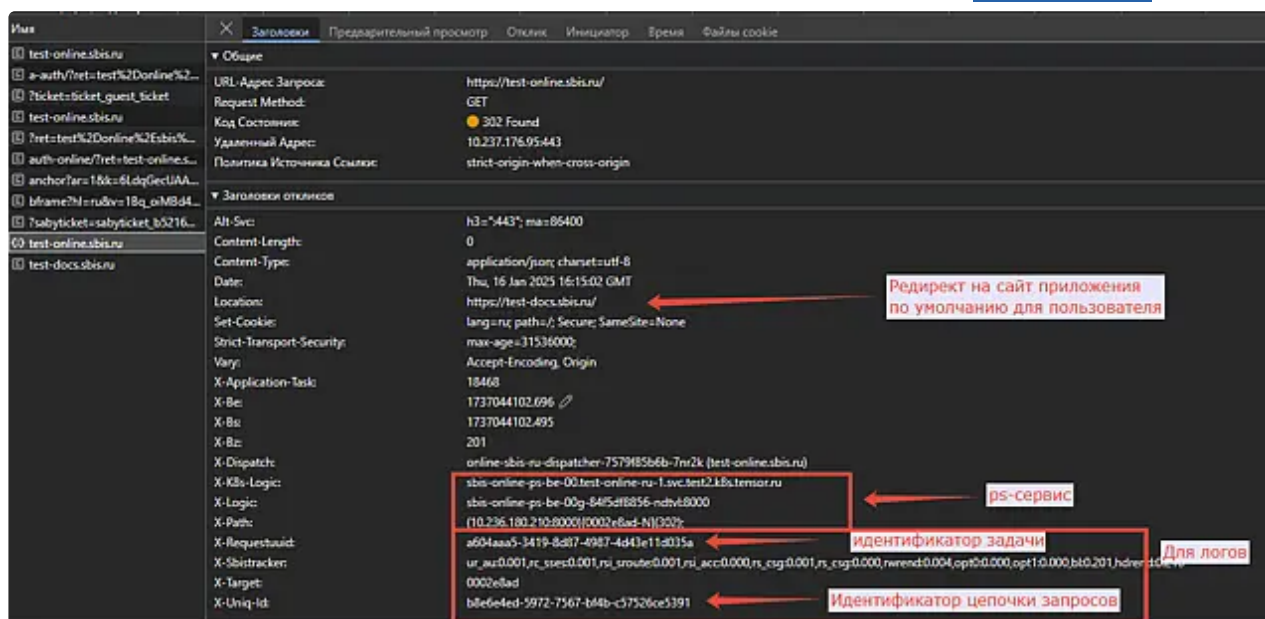


Рис. 10

Понять, что редирект сделала конфигурация можно, построив логи по x-uniq-id или x-requestuid запроса.

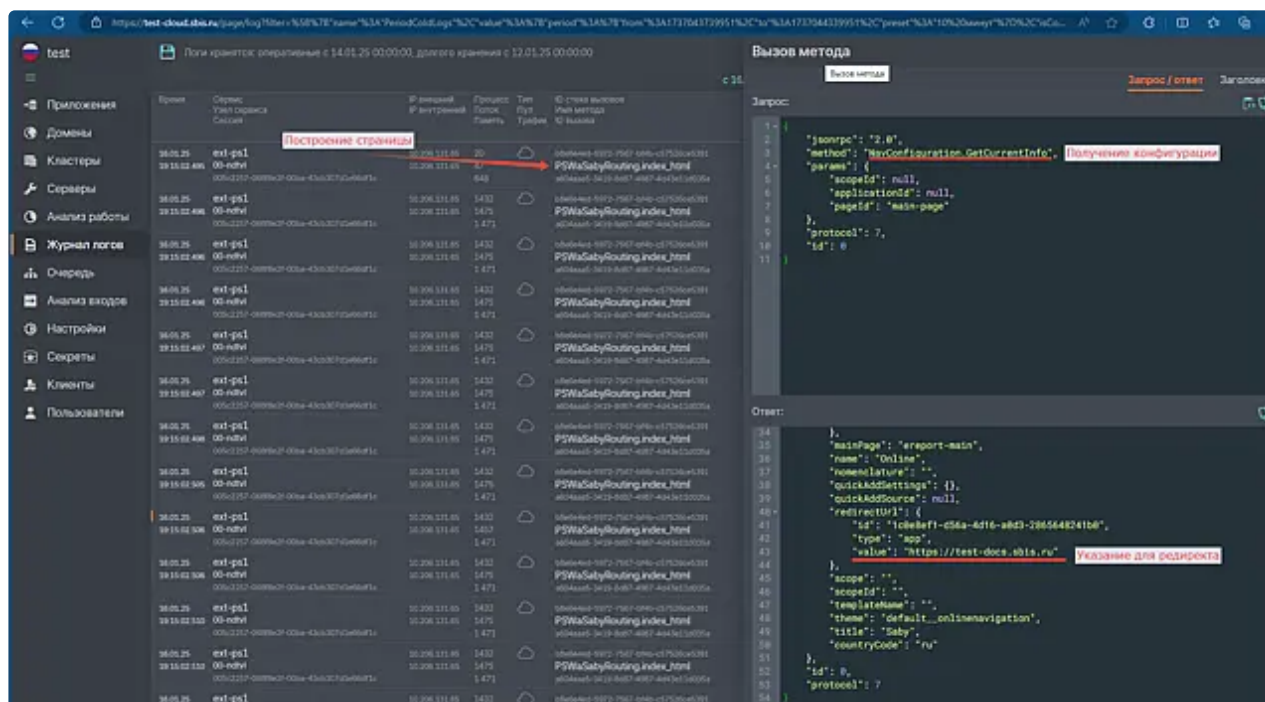


Рис. 11

Таким образом, для понимания причины редиректа нужно анализировать заголовки ответа:

- Location - адрес редиректа
- X-Path - кто выполнил запрос или сделал редирект
- X-Logic - экземпляр БЛ, который выполнил запрос
- X-Reason, X-Reason-RC - причины редиректа на диспетчере
- X-Uniq-id, X-Requestuuid - идентификаторы запроса для поиска в логах.

Подробнее про заголовки написано в [статье](#).

Как сделать редиректы на БЛ и PS-сервисе

БЛ сервис

На бизнес-логике редирект реализуется через генерацию исключения `grpc::GenericHttpWarning` с `http-кодом` редиректа. В контекст `icsCUSTOM_HEADERS` нужно установить значение `Location`.

Пример на C++:

```
1 Session::Set( icsCUSTOM_HEADERS, L"Location: " + url );
2 SBIS_THROW_EXCEPTION( rpc::GenericHttpWarning( 302, L"302 Found" ) );
```

Пример на Python:

```
1 sbis.Session.Set(sbis.WebServerContextKey.icsCUSTOM_HEADERS, "Location: " + url)
2 raise sbis.GenericHttpWarning('302', 302, '302')
```

PS сервис

На ps-сервисе в js-коде нужно получить объект `response`, и вызвать у него метод `redirect`:

```
1 const res = process.domain.res;
2 res.redirect(url);
```